Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

SKILL BASED EVALUATION REVIEW

Code-Mixed (Tanglish/Hinglish) Text: Language Identification, Normalization and N-gram/LSTM Language Modelling

25AI2008 Multilingual AI

Team Members

Angel Sweety P (5051) | Shally S (5078)
K Kokila Dharshini (5037) | Selvarani S (5088)

Course Faculty

<<Faculty Name, Designation>>

Division of Artificial Intelligence and Machine Learning
School of Computer Science and Technology
October 2026

Problem Statement & Objectives

Problem Statement

What: Tanglish (Tamil-English) and Hinglish (Hindi-English) are typed in Roman letters with English words mixed in. NLP tools built for one language and one script fail on this text.

Why it is hard: no standard spelling (iruku / irukku, hai / haiii), ambiguous words and language boundaries, names and acronyms, very few gold-annotated corpora and many unseen word combinations.

Who benefits: search, moderation, keyboard / chat prediction and analytics for users of Romanized Indian languages.

Objectives

1. **Token-level language identification**

   Success: word accuracy and macro-F1 on held-out gold data; char n-gram + LogReg vs BiLSTM

2. **Consistent code-mixed normalization**

   Success: WER vs three human references (Hinglish); spelling only, no translation

3. **Statistical vs neural language modelling**

   Success: perplexity and top-1 / top-5 next-word accuracy, same corpus and test positions

Literature Survey

Ref	Authors & Year	Method / Model	Gap / Limitation
[1]	**Sheth, Beniwal & Singh, 2025** Findings of EMNLP 2025	COMI-LINGUA: expert-annotated Hindi-English code-mixed dataset (3 annotators per instance); 5 tasks incl. token-level LID and benchmarks with fine-tuned LLMs	Hindi-English only: no Tamil-English counterpart; no lightweight from-scratch baselines
[2]	**Barman, Das, Wagner & Foster, 2014** CALCS workshop, pp. 13-23	Word-level language identification on Bengali-English-Hindi Facebook comments; preliminary experiments	Work in progress; no Tamil; LID only (no normalization or language model)
[3]	**Han & Baldwin, 2011** ACL 2011, pp. 368-378	Lexical normalisation of ill-formed out-of-vocabulary words in Twitter / SMS text (dictionary lookup combined with other methods)	Targets English short text; not designed for Romanized Tamil / Hindi mixed with English
[4]	**Pratapa et al., 2018** ACL 2018, pp. 1543-1553	RNN language model for code-mixing trained with synthetic data from Equivalence Constraint Theory	Focus on synthetic-data curriculum for perplexity; no LID or normalization of noisy Romanized text

Research gap: The surveyed work handles one task or one language pair at a time. This project covers Tanglish and Hinglish in one pipeline (language ID, normalization, language models) and compares N-gram vs LSTM under one protocol, with explicit gold / silver label provenance.

Proposed Methodology

1

Data Collection

COMI-LINGUA (Hinglish) and Tanglish-Corpus-185k, both CC-BY-4.0

2

Preprocessing

NFKC clean-up, typed tokenizer, de-duplication, split by source

3

Tokenization & Embeddings

Char n-grams + TF-IDF; word and char-CNN embeddings; LM vocabulary

4

Model Training

LogReg and BiLSTM taggers; KN trigram; LSTM LM, all from scratch

5

Evaluation

Accuracy / macro-F1, WER, perplexity, top-1 / top-5 on held-out data

Workflow Summary

Romanized Tanglish / Hinglish sentence → typed tokenizer → token-level language ID (rules for numbers, punctuation, URLs, emoji + char n-gram LogReg or BiLSTM) → normalizer (rules + language-aware dictionaries) → LM tokens → trigram-KN and LSTM next-word suggestions and perplexity.

One pipeline class serves the Streamlit app, CLI and evaluation. Module owners: <<to be filled by the team>>

Dataset & Languages

2

Languages covered: Tanglish (Tamil-English) and Hinglish (Hindi-English)

213,892

Sentences in the LM corpora: 185,601 Tanglish + 28,291 Hinglish

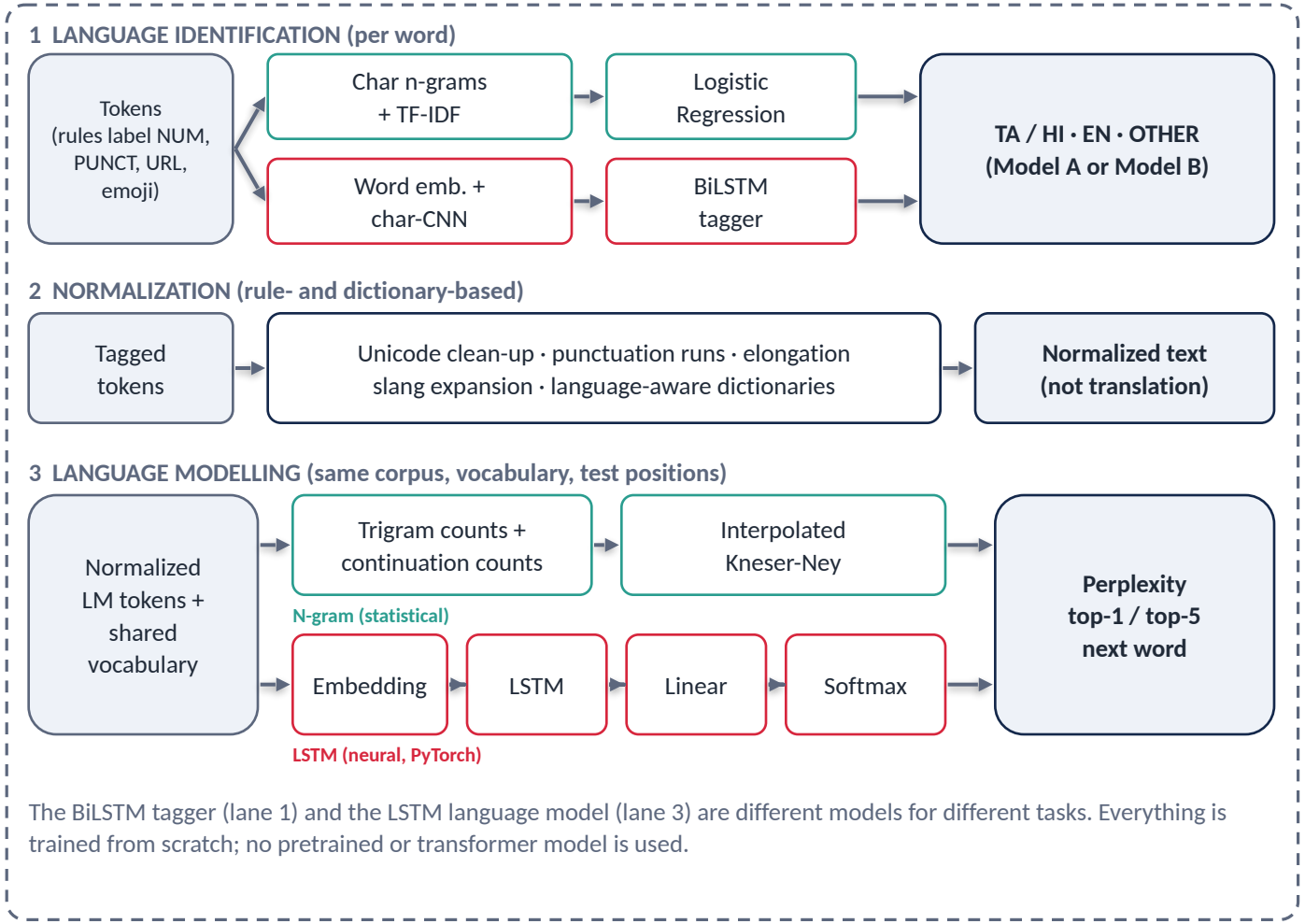
76 / 14 / 10

Train / Val / Test (%), Tanglish LM corpus. Hinglish LM corpus: 74 / 8 / 17

Source & Preparation

- **Hinglish:** COMI-LINGUA (LingoIITGN, CC-BY-4.0, huggingface.co/datasets/LingoIITGN/COMI-LINGUA); Roman-script sentences only; 3 annotators per sentence give token tags and normalized references. LID: 7,191 / 799 / 750 sentences.
- **Tanglish:** Tanglish-Corpus-185k (vishnu-n, CC-BY-4.0): 185,973 sentences from YouTube, Reddit and DravidianCodeMix; 185,670 after de-duplication; split by source (video / subreddit).
- **Labels:** Hinglish tokens are GOLD (3-annotator majority). Tanglish tokens are SILVER (lexicon rules, not human); evaluated on 120 hand-reviewed sentences, not independently validated.
- **Conventions:** TA / HI, EN, OTHER (names, acronyms, handles, noise); NUM and PUNCT by rules; numbers → <num>; native-script sentences removed.

Model Architecture



The BiLSTM tagger (lane 1) and the LSTM language model (lane 3) are different models for different tasks. Everything is trained from scratch; no pretrained or transformer model is used.

Key Components

- LID: char n-gram TF-IDF + LogReg (context-free) vs word + char-CNN + BiLSTM (sentence context)
- Normalization: rules + learned (Hinglish) / curated (Tanglish) dictionaries
- LM: KN trigram (NumPy) vs embedding → LSTM → linear → tied softmax
- No transformers, no pretrained weights

Hyperparameters

BiLSTM tagger: word emb 64, char-CNN 32-d (k = 3, 5), 128 hidden x 2, dropout 0.3, Adam 2e-3, 10 epochs

LSTM LM: Tanglish 256 / 512, dropout 0.3, 10 epochs; Hinglish 200 / 400, dropout 0.4, early stop at epoch 25; Adam 2e-3, clip 1.0

KN trigram: discount D = 0.9 (tuned on validation)

Implementation

1 Language & Libraries

Python 3.14.3 · PyTorch 2.14.0 · scikit-learn 1.9.1 · NumPy 2.4.0 · pandas 3.0.3 · wordfreq · joblib. KN trigram model, tokenizer and metrics written from scratch (no NLTK).

2 Models & Frameworks

No pretrained models: all models trained from scratch. Char n-gram + LogReg, BiLSTM tagger, KN trigram, LSTM LM (tied softmax). Streamlit 1.62.0 dashboard (Analyze, Batch, Evaluation, About); pytest: 25 tests passed.

3 Environment & Hardware

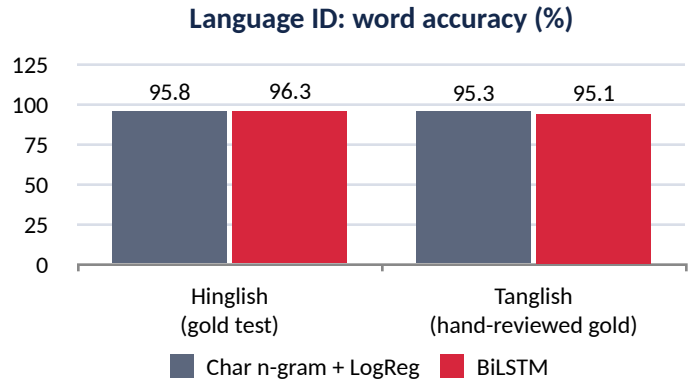
Local development on an Apple M4 laptop, 16 GB, macOS; PyTorch MPS (Apple GPU) backend. Data, models and results stored locally. LSTM training: 233 s (Hinglish), 1,206 s (Tanglish).

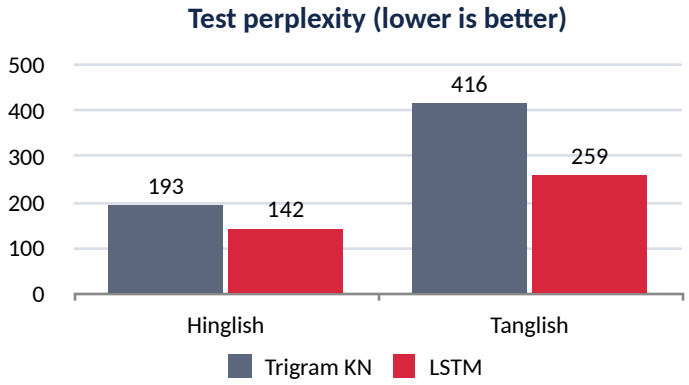
4 Modules Developed

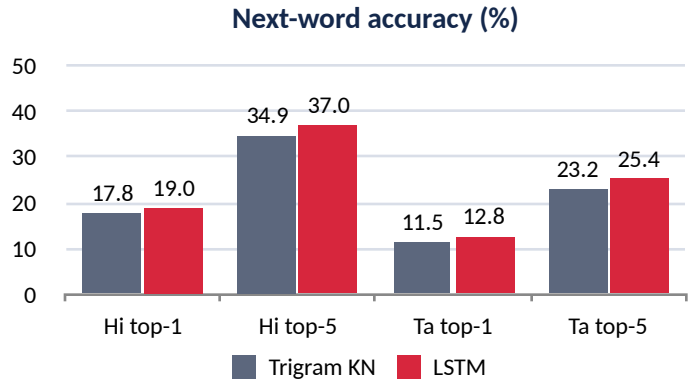
M1 language ID · M2 normalization · M3 trigram LM · M4 LSTM LM · pipeline.py + Streamlit app.py · evaluation.py

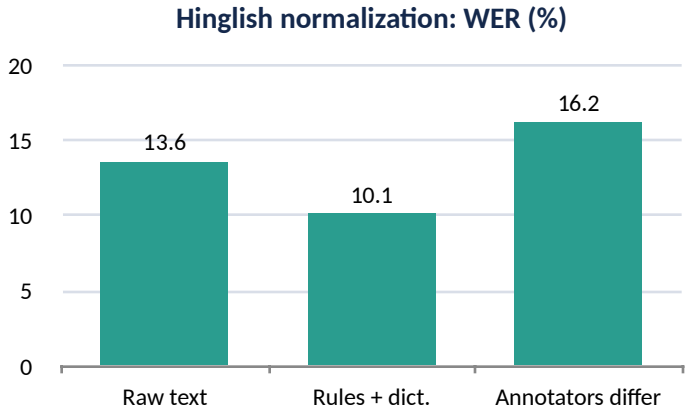
Owner per module: <<to be filled by the team>>

Results & Evaluation







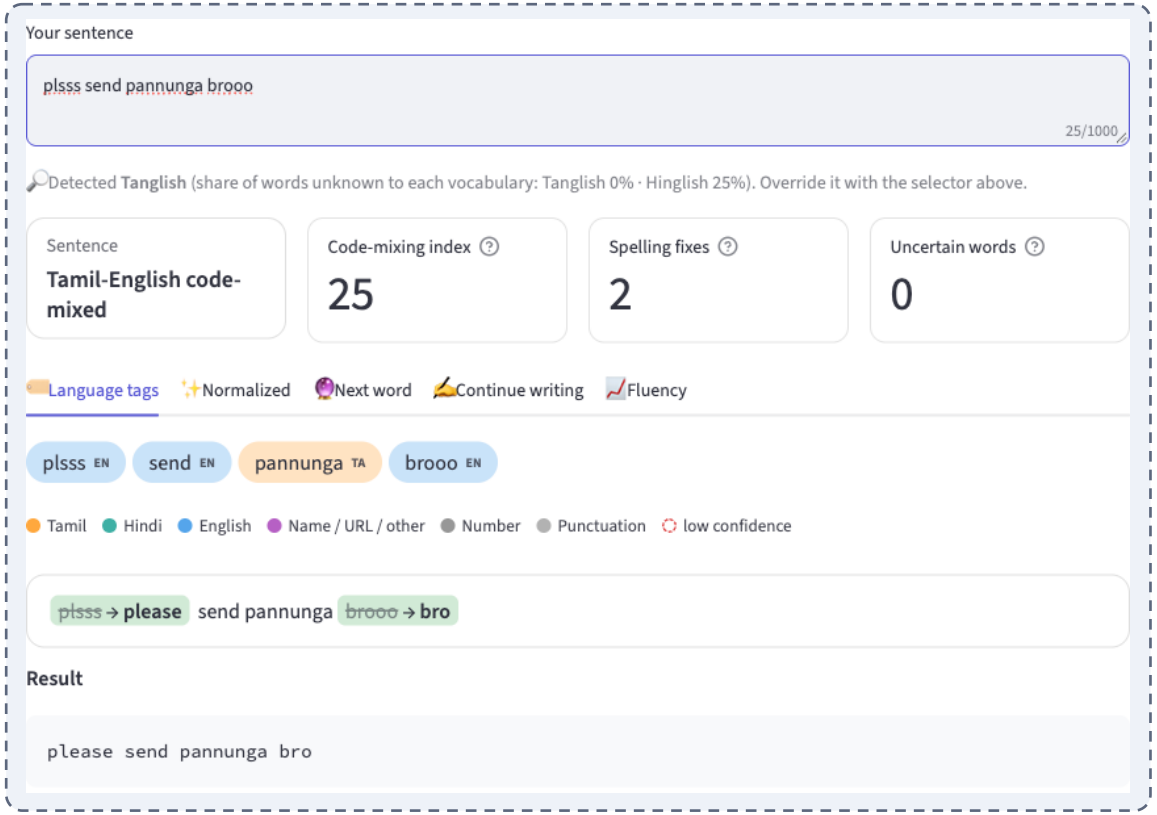


Takeaway: the LSTM lowers perplexity most (Hinglish 193 → 142, Tanglish 416 → 259); the two language-ID models score about the same. Tanglish language ID uses the 120-sentence hand-reviewed gold set.

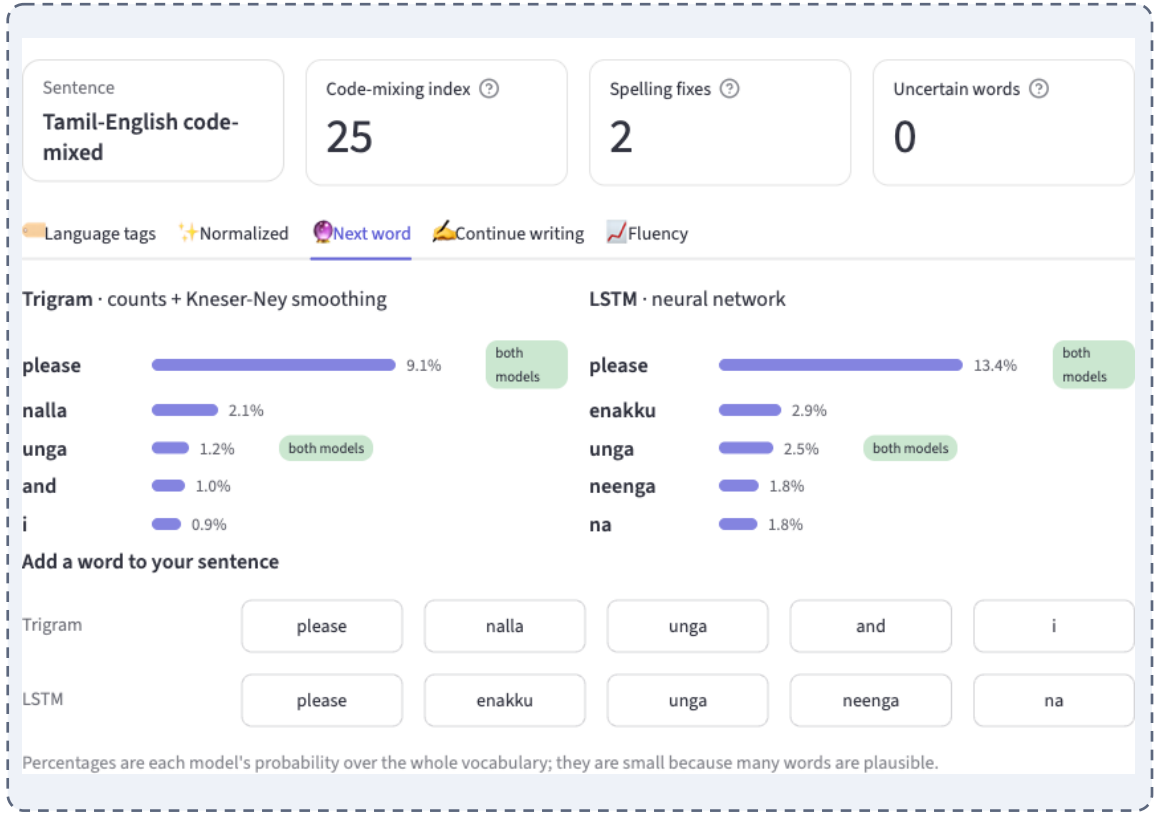
Observations

- LID: char n-gram ≈ BiLSTM (95.8 vs 96.3 Hinglish; 95.3 vs 95.1 Tanglish gold), so sentence context adds little.
- LSTM top-5 is +2.1 / +2.2 pts (Hi / Ta); top-1 only +1.2 / +1.3.
- Normalization WER 13.6% → 10.1%; the 3 annotators differ by 16.2%, so scoring is subjective.
- Metrics: accuracy, macro-F1, WER, perplexity, top-1 / top-5.
- **Limits:** Tanglish gold = 120 sentences, not independently validated; 99% silver score is inflated; names / acronyms recall about 46%; Tanglish normalization 97% uses 68 self-authored pairs.

Demo & Output Screenshots



Input: “plsss send pannunga brooo” → token-level language ID (plsss[EN] send[EN] pannunga[TA] brooo[EN]) → normalized output “please send pannunga bro”



Next-word top-5, side by side: trigram (Kneser-Ney) vs LSTM; both models rank “please” first (9.1% vs 13.4%)

Genuine screenshots of the project's Streamlit dashboard (project results/ folder and captures from the running app); cropped to the relevant panels, not edited.

Conclusion & Future Work

1 Conclusion

- Built language ID, normalization and two language models (KN trigram, LSTM) from scratch in one tested pipeline and dashboard.
- Key numbers: LID 95.8% / 96.3% (Hinglish), 95.3% / 95.1% (Tanglish gold); WER 13.6% → 10.1%; perplexity 193 → 142 and 416 → 259.
- LSTM beats the trigram on every LM metric, mainly in perplexity.
- **Limitations:** Tanglish labels are weak (silver); 120-sentence gold set not independently validated; vocabulary capped at 20k; no cross-variant transfer; one-command rebuild (run_all) not verified end to end.

2 Future Work

- Native-speaker validation and a larger Tanglish gold set
- Independent Tanglish normalization benchmark
- Better handling of names and acronyms
- Larger vocabulary and more robust GPU training
- Cross-language generalization; more Indic languages
- Context-aware, language-aware normalization; verify the full rebuild

Thank You

Questions & Discussion